50

Andrew 文件系统（AFS）

Andrew 文件系统（Andrew File System）是卡内基-梅隆大学（Carnegie-Mellon University，CMU）在 20 世纪 80 年代[1]推出的 [H+88]。这个项目由卡内基-梅隆大学著名的 M. Satyanarayanan 教授（简称“Satya”）领导，主要目标很简单：**可扩展性**（scale）。具体来说，怎样设计一个分布式文件系统（distributed file system），才能让一台服务器支持尽可能多的客户端?

有意思的是, 设计和实现的很多方面都会影响可扩展性。其中最重要的是客户端和服务器之间**协议**（protocol）的设计。例如在网络文件系统（Network File System，NFS）中，协议迫使客户端定期向服务器核对缓存（cache）的内容是否发生了变化；由于每次核对都要消耗 CPU、网络带宽（bandwidth）等服务器资源，这样频繁的核对会限制一台服务器能够响应的客户端数量，从而限制可扩展性。

AFS 和 NFS 的另一个不同之处在于, AFS 从一开始就把用户可以看到的合理行为当作头等大事。在 NFS 中，缓存一致性（cache consistency）很难描述清楚，因为它直接取决于底层的实现细节, 包括客户端缓存的超时（timeout）时间。而在 AFS 中, 缓存一致性（cache consistency）简单易懂：打开文件时，客户端一般会从服务器拿到最新的一致副本。

50.1 AFS 第 1 版

我们将讨论 AFS 的两个版本 [H+88, S+85]。第一个版本（我们称之为 AFSv1，不过最初的系统其实叫作 ITC 分布式文件系统 [S+85]）已经具备了一些基本设计，但可扩展性没有达到预期, 设计者于是重新设计, 得到了最终的协议, 我们称之为 AFSv2, 或者就叫 AFS [H+88]。下面先讨论第一个版本。

AFS 所有版本的一个基本原则, 是在访问文件的客户端机器的**本地磁盘**（local disk）上做**整文件缓存**（whole-file caching）。你 `open()` 一个文件时，整个文件（如果存在的话）都会从服务器取回，存成你本地磁盘上的一个文件。应用程序之后的 `read()` 和 `write()` 操作都被重定向（redirect）到存放该文件的本地文件系统（file system）；因此，这些操作不需要网络通信，速度很快。最后，在 `close()` 时，文件（如果被修改过）会被刷回服务

[1]虽然最初叫“Carnegie-Mellon University”，但 CMU 后来去掉了连字符，于是有了现在的写法“Carnegie Mellon University”。由于 AFS 源自 80 年代初的工作，我们沿用 CMU 最初带连字符的写法。如果你对极其无聊的细枝末节感兴趣, 可以到 https://www.quora.com/When-did-Carnegie-Mellon-University-remove-the-hyphen-in-the-university-name 看看。

`TestAuth`	检查文件是否发生了变化（用于验证缓存的条目）
`GetFileStat`	获取文件的 stat 信息
`Fetch`	取回文件的内容
`Store`	把这个文件存到服务器上
`SetFileStat`	设置文件的 stat 信息
`ListDir`	列出目录的内容

图 50.1：**AFSv1 协议要点**

器。注意它和 NFS 的明显区别：NFS 缓存的是块（block）而不是整个文件（当然 NFS 也可以把整个文件的每一块都缓存下来），而且缓存在客户端的内存里，而不是本地磁盘上。

我们来看得更细一点。客户端应用程序第一次调用 `open()` 时，AFS 的客户端代码（AFS 的设计者称之为 **Venus**）会向服务器发送一条 Fetch 协议消息（protocol message）。Fetch 协议消息把所需文件的完整路径名（pathname，例如 `/home/remzi/notes.txt`）传给文件服务器（file server，设计者把这组服务器叫作 **Vice**），服务器随后遍历这个路径名，找到所需的文件，再把整个文件发回客户端。接着，客户端代码把文件缓存在客户端的本地磁盘上（也就是把它写到本地磁盘）。前面说过，在 AFS 中，之后的 `read()` 和 `write()` 系统调用（system call）完全在本地进行（不和服务器通信），只是被重定向到文件的本地副本。由于 `read()` 和 `write()` 调用的行为和对本地文件系统的调用一样，某个块一旦被访问，也可能被缓存在客户端内存里。因此，AFS 也会用客户端内存来缓存它本地磁盘上那些块的副本。最后，用完文件后，AFS 客户端检查文件是否被修改过（即是否以写方式打开过）；如果是，就用一条 Store 协议消息把新版本刷回服务器，把整个文件连同路径名发给服务器永久保存。

下次访问这个文件时，AFSv1 的效率就高得多了。具体来说，客户端代码先用 TestAuth 协议消息联系服务器，确认文件是否发生了变化。如果没有变化，客户端就使用本地缓存的副本，避免了一次网络传输，从而提高了性能。图 50.1 列出了 AFSv1 的一些协议消息。注意，这个早期版本的协议只缓存文件内容；目录（directory）之类的东西则只保存在服务器上。

提示：先测量，再构建（Patterson 定律）

我们的导师之一 David Patterson（以精简指令集计算 RISC 和廉价磁盘冗余阵列 RAID 闻名）总是鼓励我们：在构建新系统来解决某个问题之前，先测量现有系统，证明问题确实存在。用实验证据而不是直觉，就能把构建系统的过程变成一件更科学的事。这样做还有一个附带的好处：在改进版本开发出来之前，你就得想清楚究竟该怎样测量这个系统。等你最终着手构建新系统时，有两件事会因此变得更好：第一，你有证据表明自己在解决一个真实存在的问题；第二，你已经有了一套测量新系统的方法，可以用来证明它确实比现有技术更好。这就是我们说的 **Patterson 定律**（Patterson's Law）。

 © 2008–23, ARPACI-DUSSEAU

50.2 第 1 版的问题

AFS 第一版的几个关键问题，促使设计者重新思考他们的文件系统。为了详细研究这些问题，AFS 的设计者花了大量时间测量现有的原型，找出问题所在。这样的实验是好事：**测量**（measurement）是理解系统如何工作、如何改进的关键，所以获得具体、可靠的数据，是构建系统的必要环节。在研究中，作者们发现 AFSv1 有两个主要问题：

- **路径遍历的开销太高**：执行 Fetch 或 Store 协议请求（protocol request）时，客户端把完整的路径名传给服务器，例如 `/home/remzi/notes.txt`。为了访问这个文件，服务器必须完整地遍历路径名：先在根目录（root directory）中查找 `home`，再在 `home` 中查找 `remzi`，依此类推，沿路径一路往下，直到最终找到所需的文件。AFS 的设计者发现，大量客户端同时访问服务器时，服务器把大量 CPU 时间都花在了沿目录路径往下走上。
- **客户端发出的 TestAuth 协议消息太多**：和 NFS 发出过多的 GETATTR 协议消息很像，AFSv1 也产生了大量流量，用 TestAuth 协议消息检查本地文件（或者它的 stat 信息）是否有效。于是，服务器把大量时间都花在告诉客户端能不能使用它们缓存的文件副本上。而大多数时候，答案都是文件没有变化。

AFSv1 其实还有另外两个问题：服务器之间的负载不均（load imbalance）；服务器为每个客户端单独使用一个进程（process），从而带来了上下文切换（context switch）和其他开销。负载不均（load imbalance）的问题靠引入**卷**（volume）来解决：管理员可以在服务器之间移动卷来平衡负载；上下文切换的问题，AFSv2 改用线程（thread）而不是进程来构建服务器，也解决了。不过，限于篇幅，这里我们只关注上面那两个限制系统规模的主要协议问题。

50.3 改进协议

上面两个问题限制了 AFS 的可扩展性：服务器的 CPU 成了系统的瓶颈，每台服务器只能服务 20 个客户端，再多就会过载。服务器收到了太多的 TestAuth 消息，而在收到 Fetch 或 Store 消息时，又在遍历目录层次结构（directory hierarchy）上花了太多时间。于是，AFS 的设计者面临这样一个问题：

> 关键问题：如何设计可扩展的文件协议
> 应该怎样重新设计协议，才能把与服务器的交互次数降到最少，也就是说，怎样减少 TestAuth 消息的数量？此外，怎样设计协议，才能让这些与服务器的交互变得高效？把这两个问题都解决了，新协议就能让 AFS 的可扩展性大大提高。

50.4 AFS 第 2 版

AFSv2 引入了**回调**（callback）的概念，以减少客户端与服务器之间的交互次数。回调就是服务器对客户端的一个承诺：客户端缓存的某个文件被修改时，服务器会通知客户端。系统里加入了这份**状态**（state）之后，客户端就不必再联系服务器来确认缓存的文件是否仍然

 © 2008–23, ARPACI-DUSSEAU

有效。相反，它会认为文件一直有效，直到服务器告诉它并非如此为止；这和**轮询**（polling）与**中断**（interrupts）的区别正好可以类比。

AFSv2 还引入了**文件标识符**（file identifier，**FID**）的概念，它类似于 NFS 的**文件句柄**（file handle），用来代替路径名指定客户端感兴趣的文件。AFS 中的 FID 由卷标识符（volume identifier）、文件标识符（file identifier）和一个“uniquifier”组成（uniquifier 让文件被删除后，卷 ID 和文件 ID 可以被重用）。这样，客户端就不再把完整的路径名发给服务器、让服务器沿路径名查找所需的文件，而是由客户端自己一段一段地遍历路径名，并把结果缓存下来，从而有望减轻服务器的负载。

例如，假设客户端访问文件 `/home/remzi/notes.txt`，而 `home` 是挂载（mount）到 / 上的 AFS 目录，即 / 是本地根目录，`home` 及其下面的内容都在 AFS 中。这时，客户端会先 Fetch `home` 的目录内容，放进本地磁盘缓存（local-disk cache），并在 `home` 上建立一个回调。然后，客户端会 Fetch 目录 `remzi`，放进本地磁盘缓存（local-disk cache），并在 `remzi` 上建立一个回调。最后，客户端 Fetch `notes.txt`，把这个普通文件缓存到本地磁盘上，建立一个回调，然后把一个文件描述符（file descriptor）返回给调用它的应用程序。图 50.2 对此做了总结。

不过，和 NFS 的关键区别在于，每取回一个目录或文件，AFS 客户端都会在服务器上建立一个回调，从而确保服务器会在客户端缓存的状态发生变化时通知客户端。好处是显而易见的：虽然第一次访问 `/home/remzi/notes.txt` 会产生很多客户端与服务器之间的消息（如上所述），但它同时为所有目录以及文件 `notes.txt` 建立了回调，因此之后的访问完全在本地进行，根本不需要与服务器交互。所以，在文件已缓存在客户端这种常见情况（common case）下，AFS 的行为和基于本地磁盘的文件系统几乎一模一样。如果多次访问同一个文件，第二次访问应该和访问本地文件一样快。

50.5 缓存一致性

讨论 NFS 时，我们考虑了缓存一致性（cache consistency）的两个方面：**update visibility** 和 **cache staleness**。对于 update visibility，问题是：服务器什么时候会更新为文件的新版本？对于 cache staleness，问题是：服务器有了新版本之后，要过多久客户端才能看到新版本，而不是之前缓存的旧副本？

由于有回调和整文件缓存（whole-file caching），AFS 提供的缓存一致性（cache consistency）很容易描述和理解。需要考虑两种重要情况：不同机器上的进程之间的一致性，以及同一台机器上的进程之间的一致性。

在不同的机器之间，AFS 让更新在服务器上可见、并让缓存的副本失效，这两件事在同一时刻发生，也就是更新后的文件被关闭的时候。客户端打开一个文件，然后写入（也许写了很多次）。最终关闭时，新文件被刷到服务器上（于是变得可见）。这时，服务器会“打破”所有缓存了副本的客户端的回调；具体做法是联系每个客户端，通知它在这个文件上的回调已经失效。这一步确保客户端不会再读到文件的过时副本；这些客户端之后再打开这个文件时，需要从服务器重新取回新版本，同时也会在新版本上重新建立回调。

 © 2008–23, ARPACI-DUSSEAU

客户端（C_1）	**服务器**
fd = open(“/home/remzi/notes.txt”, ⋯); 发送 Fetch (home FID, “remzi”)	
	接收 Fetch 请求 在 home 目录中查找 remzi 在 remzi 上建立 callback(C_1) 返回 remzi 的内容和 FID
接收 Fetch 回复 把 remzi 写入本地磁盘缓存 记录 remzi 的回调状态 发送 Fetch (remzi FID, “notes.txt”)	
	接收 Fetch 请求 在 remzi 目录中查找 notes.txt 在 notes.txt 上建立 callback(C_1) 返回 notes.txt 的内容和 FID
接收 Fetch 回复 把 notes.txt 写入本地磁盘缓存 记录 notes.txt 的回调状态 对缓存的 notes.txt 执行本地 `open()` 把文件描述符返回给应用程序	
read(fd, buffer, MAX); 对缓存的副本执行本地 `read()`	
close(fd); 对缓存的副本执行本地 `close()` 如果文件被修改过，就刷回服务器	
fd = open(“/home/remzi/notes.txt”, ⋯); Foreach dir (home, remzi) if (callback(dir) == VALID) 用本地副本执行 lookup(dir) else Fetch（同上） if (callback(notes.txt) == VALID) 打开本地缓存的副本 返回指向它的文件描述符 else Fetch（同上），然后打开并返回 fd	

图 50.2：**读取文件：客户端和文件服务器的操作**

补充：缓存一致性不是万能药

讨论分布式文件系统时，人们很看重文件系统提供的缓存一致性（cache consistency）。然而，这种基本的一致性并不能解决多个客户端访问文件时的所有问题。例如，如果你在构建一个代码仓库，有多个客户端在签入和签出代码，就不能单靠底层文件系统替你完成所有工作；你得显式地使用**文件级锁**（file-level locking），保证发生这类并发访问（concurrent access）时结果是“正确”的。事实上，任何真正在意并发更新（concurrent update）的应用程序，都会额外加上处理冲突的机制。本章和上一章介绍的基本一致性，主要用于日常使用，也就是说，用户登录到另一台客户端时，希望在那里看到自己文件的某个合理版本。对这些协议寄予更多期望，只会让自己遭遇失败、失望，以及满含泪水的挫败感。

 © 2008–23, ARPACI-DUSSEAU

客户端$_1$			客户端$_2$		服务器	说明
P_1	P_2	缓存	P_3	缓存	磁盘	
open(F)		-		-	-	文件被创建
write(A)		A		-	-	
close()		A		-	A	
	open(F)	A		-	A	
	read() → A	A		-	A	
	close()	A		-	A	
open(F)		A		-	A	
write(B)		B		-	A	
	open(F)	B		-	A	本地进程
	read() → B	B		-	A	立即看到写入
	close()	B		-	A	
		B	open(F)	A	A	远程进程
		B	read() → A	A	A	看不到写入……
		B	close()	A	A	
close()		B		~~A~~	B	……直到 close()
		B	open(F)	B	B	发生之后
		B	read() → B	B	B	
		B	close()	B	B	
		B	open(F)	B	B	
open(F)		B		B	B	
write(D)		D		B	B	
		D	write(C)	C	B	
		D	close()	C	C	
close()		D		~~C~~	D	
		D	open(F)	D	D	P_3 很不走运：
		D	read() → D	D	D	最后写入者胜出
		D	close()	D	D	

图 50.3：**缓存一致性时间线**

对于同一台机器上的进程，AFS 在这个简单模型之外做了一个例外。在这种情况下，对文件的写入会立即对本地的其他进程可见（也就是说，进程不必等到文件关闭才能看到最新的更新）。这让单台机器的使用行为和你预期的完全一样，因为这种行为基于典型的 UNIX 语义。只有换到另一台机器上时，你才能察觉到 AFS 更一般的一致性机制。

还有一种跨机器的情况很有意思，值得进一步讨论。具体来说，在不同机器上的进程同时修改一个文件这种少见的情况下，AFS 自然而然地采用了所谓**最后写入者胜出**（last writer wins）的做法（也许应该叫**最后关闭者胜出**，last closer wins）。也就是说，哪个客户端最后调用 `close()`，它就最后更新服务器上的整个文件，于是它的文件就成为“胜出”的文件，也就是留在服务器上供其他人查看的那个文件。结果得到的文件要么完全由这个客户端生成，要么完全由那个客户端生成。注意它和 NFS 这种基于块的协议（block-based protocol）的区别：在 NFS 中，每个客户端更新文件时，各个块的写入都可能被刷到服务器上，因此服务器上最终的文件可能是两个客户端更新的混合。在很多情况下，这样混合的文件输出没有多大意义；想象一下两个客户端分块修改一张 JPEG 图片，写入混在一起的结果多半不是一个合法的 JPEG。

 © 2008–23, ARPACI-DUSSEAU

图 50.3（第 6 页）的时间线展示了几种不同的场景。各列分别是：客户端$_1$ 上的两个进程（P_1 和 P_2）及其缓存状态，客户端$_2$ 上的一个进程（P_3）及其缓存状态，以及服务器，它们都在操作同一个文件，这个文件的名字毫无想象力地叫作 F。对于服务器，图中只给出左边的操作完成之后文件的内容。从头读一遍，看看你能不能理解每次读取为什么返回那样的结果。如果卡住了，右边的说明一栏可以帮到你。

50.6 崩溃恢复

从上面的描述中，你也许已经感觉到，崩溃恢复（crash recovery）要比 NFS 复杂。你说得没错。例如，设想有一小段时间，服务器（S）无法联系某个客户端（C1），比如客户端 C1 正在重启。在 C1 不可用期间，S 可能试图向它发送一条或多条回调召回消息；例如，设想 C1 的本地磁盘上缓存了文件 F，然后 C2（另一个客户端）更新了 F，于是 S 向所有缓存了这个文件的客户端发送消息，让它们把文件从本地缓存中移除。由于 C1 在重启时可能错过这些关键消息，它在重新加入系统时，应该把自己缓存的所有内容都视为可疑。因此，下次访问文件 F 时，C1 应该先用 TestAuth 协议消息询问服务器，它缓存的文件 F 副本是否仍然有效；如果有效，C1 就可以使用；如果无效，C1 就应该从服务器取回较新的版本。

服务器崩溃后的恢复也更复杂。问题在于回调是保存在内存里的；因此，服务器重启后，并不知道哪台客户端机器拥有哪些文件。所以，服务器重启时，它的每个客户端都必须意识到服务器崩溃过，并把自己缓存的所有内容都视为可疑，（像上面那样）先重新确认文件的有效性再使用。可见，服务器崩溃是一件大事，必须确保每个客户端及时得知这次崩溃，否则就有客户端访问到过时文件的风险。实现这种恢复的方法有很多；例如，服务器重新运行起来时，向每个客户端发送一条消息（说“别相信你缓存的内容！”），或者让客户端用所谓的**心跳**（heartbeat）消息定期检查服务器是否还活着。可以看到，构建一个可扩展性更好、更合理的缓存模型是有代价的；而在 NFS 中，客户端几乎察觉不到服务器崩溃。

工作负载	**NFS**	**AFS**	$\frac{\textbf{AFS}}{\textbf{NFS}}$
1. 小文件，顺序读	$N_s \cdot L_{\text{net}}$	$N_s \cdot L_{\text{net}}$	1
2. 小文件，顺序重读	$N_s \cdot L_{\text{mem}}$	$N_s \cdot L_{\text{mem}}$	1
3. 中等文件，顺序读	$N_m \cdot L_{\text{net}}$	$N_m \cdot L_{\text{net}}$	1
4. 中等文件，顺序重读	$N_m \cdot L_{\text{mem}}$	$N_m \cdot L_{\text{mem}}$	1
5. 大文件，顺序读	$N_L \cdot L_{\text{net}}$	$N_L \cdot L_{\text{net}}$	1
6. 大文件，顺序重读	$N_L \cdot L_{\text{net}}$	$N_L \cdot L_{\text{disk}}$	$\frac{L_{\text{disk}}}{L_{\text{net}}}$
7. 大文件，单次读	L_{net}	$N_L \cdot L_{\text{net}}$	N_L
8. 小文件，顺序写	$N_s \cdot L_{\text{net}}$	$N_s \cdot L_{\text{net}}$	1
9. 大文件，顺序写	$N_L \cdot L_{\text{net}}$	$N_L \cdot L_{\text{net}}$	1
10. 大文件，顺序覆盖写	$N_L \cdot L_{\text{net}}$	$2 \cdot N_L \cdot L_{\text{net}}$	2
11. 大文件，单次写	L_{net}	$2 \cdot N_L \cdot L_{\text{net}}$	$2 \cdot N_L$

图 50.4：**比较：AFS 与 NFS**

 © 2008–23, ARPACI-DUSSEAU

50.7 AFSv2 的规模与性能

新协议就位之后，设计者对 AFSv2 做了测量，发现它的可扩展性比原来的版本好得多。事实上，每台服务器可以支持大约 50 个客户端（而不只是 20 个）。另一个好处是，客户端的性能往往非常接近本地性能，因为在常见情况下，所有文件访问都在本地进行；文件读取通常走本地磁盘缓存（local disk cache），还可能走本地内存。只有客户端创建新文件或写入已有文件时，才需要向服务器发送 Store 消息，用新内容更新文件。

我们再把常见的文件系统访问场景和 NFS 做个比较，从而对 AFS 的性能有一些认识。图 50.4（第 7 页）给出了我们定性比较的结果。

在图中，我们针对不同大小的文件，用分析的方法考察典型的读写模式。小文件有 N_s 个块，中等文件有 N_m 个块，大文件有 N_L 个块。我们假设小文件和中等文件能放进客户端的内存；大文件能放进本地磁盘，但放不进客户端内存。

为了便于分析，我们还假设通过网络从远程服务器访问一个文件块要花 L_{net} 个时间单位，访问本地内存要花 L_{mem}，访问本地磁盘要花 L_{disk}。一般的假设是 $L_{\text{net}} > L_{\text{disk}} > L_{\text{mem}}$。

最后，我们假设第一次访问文件时不会命中任何缓存。之后的文件访问（即“重读”），只要相应的缓存容量足以容纳这个文件，我们就假设会命中缓存。

图中各列给出了某个操作（例如小文件顺序读）在 NFS 或 AFS 上大致需要的时间。最右边一列给出 AFS 与 NFS 的比值。

我们有以下几点观察。第一，在很多情况下，两个系统的性能大致相当。例如，在第一次读取文件的工作负载（workload）1、3、5 中，从远程服务器取回文件的时间占了主要部分，在两个系统上差不多。你可能以为 AFS 在这种情况下会慢一些，因为它要把文件写到本地磁盘；然而，这些写入会被本地（客户端）文件系统的缓存缓冲起来，所以这部分开销很可能被掩盖了。你可能还会以为 AFS 从本地缓存的副本读取会慢一些，原因同样是 AFS 把缓存的副本存在磁盘上。然而，AFS 在这里同样得益于本地文件系统的缓存；AFS 上的读取很可能命中客户端的内存缓存，性能和 NFS 差不多。

第二，工作负载 6 是大文件顺序重读，这时出现了一个有意思的差别。由于 AFS 有很大的本地磁盘缓存，再次访问这个文件时，它会从本地磁盘缓存中读取。而 NFS 只能把块缓存在客户端内存里；结果，如果重读一个大文件（即比本地内存还大的文件），NFS 客户端就得从远程服务器重新取回整个文件。因此，假设远程访问确实比本地磁盘慢，那么在这种情况下 AFS 比 NFS 快 $\frac{L_{\text{net}}}{L_{\text{disk}}}$ 倍。我们还注意到，NFS 在这种情况下会增加服务器的负载，这也会影响可扩展性。

第三，我们注意到，在工作负载 8、9 中，（新文件的）顺序写在两个系统上的性能应该差不多。在这种情况下，AFS 会把文件写入本地缓存的副本；文件关闭时，AFS 客户端会按照协议把写入强制写到服务器。NFS 会把写入缓冲在客户端内存中，可能因为客户端的内存压力（memory pressure）而把一些块强制写到服务器，但为了保持 NFS 关闭时刷新（flush-on-close）的一致性，在文件关闭时肯定会把它们写到服务器。你可能以为 AFS 在这里会慢一些，因为它把所有数据都写到本地磁盘。然而要知道，它是在写本地文件系统；这些写入先提交到页缓存（page cache），之后才（在后台）写到磁盘，因此 AFS 借助客户端操作系统的内存缓存提高了性能。

第四，我们注意到，AFS 在顺序覆盖写一个文件时表现更差，即工作负载 10。到目前为止，我们一直假设执行写入的工作负载也在创建新文件；而在这种情况下，文件已经存在，然后被覆盖写。对 AFS 来说，覆盖写可能是特别糟糕的情况，因为客户端先要把旧文件整个取回来，然后再把它覆盖掉。而 NFS 只会直接覆盖这些块，从而避免了最初那次（无用的）读取[2]。

最后，访问大文件中一小部分数据的工作负载，在 NFS 上的表现比在 AFS 上好得多（工作负载 7、11）。在这些情况下，AFS 协议在打开文件时会取回整个文件；可惜，之后只执行了一次很小的读或写。更糟的是，如果文件被修改了，整个文件都要写回服务器，性能影响翻倍。NFS 作为一种基于块的协议（block-based protocol），执行的 I/O 量和读写的大小成正比。

总的来说，我们看到 NFS 和 AFS 做了不同的假设，因此得到不同的性能结果也就不足为奇了。这些差别是否重要，和往常一样，取决于工作负载。

50.8 AFS 的其他改进

伯克利快速文件系统（Fast File System，FFS）问世时加入了符号链接（symbolic link）和许多其他功能；AFS 的设计者也一样，借构建系统的机会，加入了许多让系统更易于使用和管理的功能。例如，AFS 为客户端提供了真正的全局命名空间（global namespace），确保所有文件在所有客户端机器上的名字都一样。而 NFS 允许每个客户端按自己喜欢的任何方式挂载 NFS 服务器，因此只有靠约定（以及大量的管理工作），文件在各个客户端上的名字才会相似。

AFS 还认真对待安全问题，加入了认证（authenticate）用户的机制，并确保用户愿意的话，可以让一组文件保持私有。而 NFS 在很多年里对安全的支持都相当原始。

补充：工作负载的重要性

评估任何系统都面临一个挑战：选择什么样的**工作负载**（workload）。由于计算机系统的用法千差万别，可供选择的工作负载种类繁多。存储系统的设计者应该怎样判断哪些工作负载重要，从而做出合理的设计决策呢?

AFS 的设计者有测量文件系统使用情况的经验，因此对工作负载做了一些假设；特别是，他们假设大多数文件不会被频繁共享，而且会被从头到尾顺序访问。在这些假设下，AFS 的设计非常合理。

然而，这些假设并不总是成立。例如，设想一个应用程序定期往日志里追加信息。这些小规模的日志写入，往一个已有的大文件里添加少量数据，对 AFS 来说相当麻烦。还有很多其他难以处理的工作负载，例如事务（transaction）数据库中的随机更新。

要了解哪些类型的工作负载比较常见，可以参考已经做过的各种研究。这些研究都是工作负载分析的好例子 [B+91, H+11, R+00, V99]，其中也包括 AFS 的回顾性论文 [H+88]。

[2]这里我们假设 NFS 的写入都以块为大小、按块对齐；否则 NFS 客户端也得先读出这个块。我们还假设打开文件时没有使用 `O_TRUNC` 标志；如果用了，AFS 最初的打开就不会去取回这个马上要被截断的文件的内容。

此外，AFS 提供了由用户灵活管理访问控制（access control）的功能。因此，使用 AFS 时，用户可以在很大程度上控制谁能访问哪些文件。而 NFS 和大多数 UNIX 文件系统一样，对这类共享的支持要少得多。

最后，前面提到过，AFS 还加入了一些工具，让系统管理员能更简单地管理服务器。在系统管理方面，AFS 的考虑比同行领先了好几光年。

50.9 小结

AFS 向我们展示了，分布式文件系统可以用和 NFS 非常不同的方式来构建。AFS 的协议设计尤其重要：它用整文件缓存（whole-file caching）和回调把与服务器的交互降到最少，因此每台服务器能支持很多客户端，管理一个站点所需的服务器也就少了。还有许多其他特性，包括单一命名空间、安全和访问控制列表（access control list），让 AFS 用起来非常舒服。AFS 提供的一致性模型简单，易于理解和推断，不会像 NFS 那样偶尔出现奇怪的行为。

也许令人遗憾的是，AFS 很可能正在走下坡路。NFS 成了开放标准，很多厂商都支持它，它和 CIFS（基于 Windows 的分布式文件系统协议）一起主导了市场。虽然时不时还能看到 AFS 的部署（比如在一些教育机构，包括威斯康星大学），但真正能留下持久影响的，很可能是 AFS 的思想，而不是这个系统本身。事实上，NFSv4 现在加入了服务器状态（例如一条“open”协议消息），因此和 AFS 的基本协议越来越像了。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[B+91] “Measurements of a Distributed File System” by Mary Baker, John Hartman, Martin Kupfer, Ken Shirriff, John Ousterhout. SOSP ’91, Pacific Grove, California, October 1991. *An early paper measuring how people use distributed file systems. Matches much of the intuition found in AFS.*

[H+11] “A File is Not a File: Understanding the I/O Behavior of Apple Desktop Applications” by Tyler Harter, Chris Dragga, Michael Vaughn, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. SOSP ’11, New York, New York, October 2011. *Our own paper studying the behavior of Apple Desktop workloads; turns out they are a bit different than many of the server-based workloads the systems research community usually focuses upon. Also a good recent reference which points to a lot of related work.*

[H+88] “Scale and Performance in a Distributed File System” by John H. Howard, Michael L. Kazar, Sherri G. Menees, David A. Nichols, M. Satyanarayanan, Robert N. Sidebotham, Michael J. West. ACM Transactions on Computing Systems (ACM TOCS), Volume 6:1, February 1988. *The long journal version of the famous AFS system, still in use in a number of places throughout the world, and also probably the earliest clear thinking on how to build distributed file systems. A wonderful combination of the science of measurement and principled engineering.*

[R+00] “A Comparison of File System Workloads” by Drew Roselli, Jacob R. Lorch, Thomas E. Anderson. USENIX ’00, San Diego, California, June 2000. *A more recent set of traces as compared to the Baker paper [B+91], with some interesting twists.*

[S+85] “The ITC Distributed File System: Principles and Design” by M. Satyanarayanan, J.H. Howard, D.A. Nichols, R.N. Sidebotham, A. Spector, M.J. West. SOSP ’85, Orcas Island, Washington, December 1985. *The older paper about a distributed file system. Much of the basic design of AFS is in place in this older system, but not the improvements for scale. The name change to “Andrew” is an homage to two people both named Andrew, Andrew Carnegie and Andrew Mellon. These two rich dudes started the Carnegie Institute of Technology and the Mellon Institute of Industrial Research, respectively, which eventually merged to become what is now known as Carnegie Mellon University.*

[V99] “File system usage in Windows NT 4.0” by Werner Vogels. SOSP ’99, Kiawah Island Resort, South Carolina, December 1999. *A cool study of Windows workloads, which are inherently different than many of the* UNIX*-based studies that had previously been done.*

 © 2008–23, ARPACI-DUSSEAU

作业（模拟）

本节介绍 `afs.py`，这是一个简单的 AFS 模拟器，可以用来巩固你对 Andrew 文件系统（Andrew File System）工作原理的理解。详情请阅读 README 文件。

问题

1. 运行几个简单的例子，确认你能预测客户端会读到什么值。改变随机种子选项（`-s`），看看你能不能跟踪执行过程，预测出中间值以及最终存进文件的值。再改变文件数量（`-f`）、客户端数量（`-C`）和读操作比例（`-r`，取值在 0 到 1 之间），让问题更有挑战性一些。你也许还想生成稍长一点的跟踪，让交互更有意思，例如用 `-n 2` 或更大的值。
2. 现在做同样的事，看看你能不能预测出 AFS 服务器发起的每一次回调。试试不同的随机种子，并且让程序替你算出答案时（用 `-c`），一定要用较高的详细程度（例如 `-d 3`），以便看到回调在什么时候发生。你能准确猜出每次回调发生的时间吗？回调发生的确切条件是什么？
3. 和上面类似，用不同的随机种子运行，看看你能不能预测出每一步缓存的确切状态。用 `-c` 和 `-d 7` 运行，就能观察到缓存状态。
4. 现在来构造一些特定的工作负载。用 `-A oa1:w1:c1,oa1:r1:c1` 选项运行模拟。在随机调度程序（scheduler）下运行时，客户端 1 读取文件 a 可能看到哪些不同的值？（试试不同的随机种子，看看不同的结果。）在两个客户端操作所有可能的调度交错中，有多少种会让客户端 1 读到值 1，又有多少种读到值 0？
5. 现在来构造一些特定的调度。用 `-A oa1:w1:c1,oa1:r1:c1` 选项运行时，再分别使用下面这些调度：`-S 01`、`-S 100011`、`-S 011100`，以及你能想到的其他调度。客户端 1 会读到什么值？
6. 现在用这个工作负载运行：`-A oa1:w1:c1,oa1:w1:c1`，并像上面那样改变调度。用 `-S 011100` 运行时会发生什么？用 `-S 010011` 运行呢？决定文件最终值的关键是什么？

 © 2008–23, ARPACI-DUSSEAU